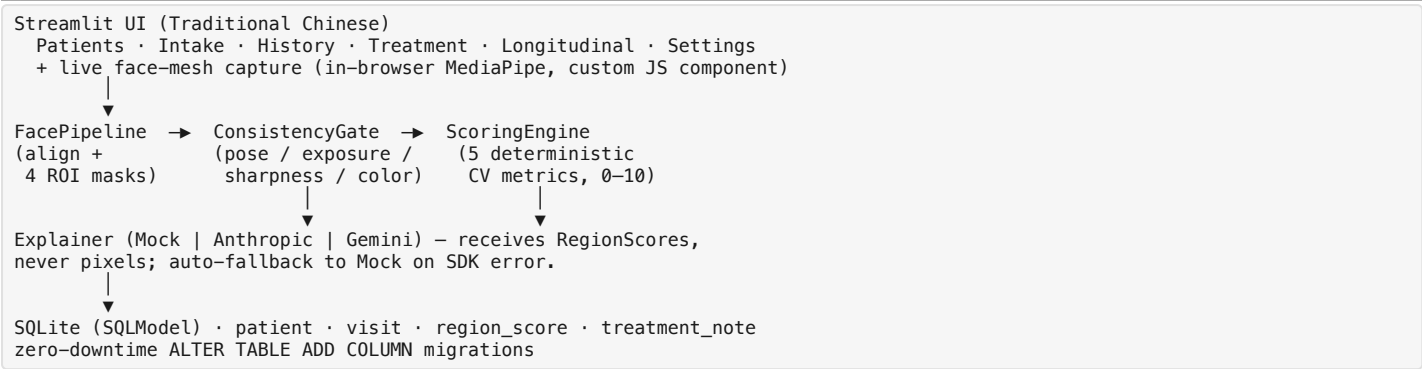


FaceTrack CRM — Technical Design Document

Author: Eric Tsou For: AI Fund Engineer in Residence — Build Challenge Date: 2026-05-19 Companion: docs/PRD.md

1. System architecture



Strict one-way flow: FacePipeline → Gate → Scoring → DB; the LLM hangs off the side as a presentation-layer convenience.

2. Imaging pipeline · src/facetrack/cv_pipeline.py

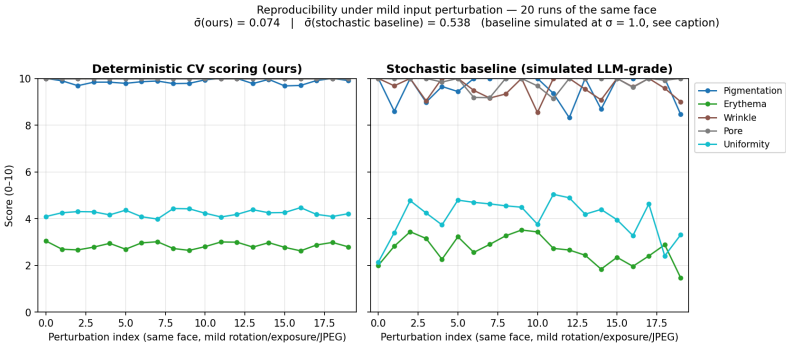
- **Model:** MediaPipe Face Landmarker (Tasks API, vendored face_landmarker.task, 3.6 MB). 478 landmarks + a 4×4 facial-transformation matrix consumed downstream for pose.
- **Alignment:** rotate around eye-centre by the eye-line angle (indices 33 / 263) so the inter-pupillary line is horizontal; the same affine matrix is applied to the landmarks in homogeneous coordinates so ROI geometry stays in aligned-image space.
- **ROIs:** four anatomical regions (LEFT_CHEEK, RIGHT_CHEEK, FOREHEAD, CHIN) defined as **polygon paths over the aligned landmarks** (forehead = 18 indices, cheeks ≈ 10 each, chin ≈ 12), then rasterised to per-region binary masks. Scoring runs strictly **inside the mask** (scoring_ratio_inside / _stat_inside), so background pixels — hair, jewellery, clothing — can never contaminate the texture metrics. Polygons were chosen over the original axis-aligned bounding boxes once it became clear that a forehead bbox bleeds into the hairline on most face shapes.
- **Per-photo CLAHE** on LAB-L of each ROI normalises *within-photo* lighting; *cross-photo* normalisation is the gate's job, not the scorer's.

3. Model reliability & explainability · src/facetrack/scoring.py

Score	Formula	Why this choice
Pigmentation	MORPH_BLACKHAT(gray, 15×15) pixel ratio > 18	Black-hat highlights small dark structures — the morphological signature of melanin spots.
Erythema	mean of LAB.a* over the ROI mask	Standard clinical proxy; a* is luminance-independent after the gate calibrates colour.
Wrinkle	Sobel-magnitude > 30 pixel ratio (post-Gaussian)	Cheap, isotropic edge-density proxy for fine-line content.
Pore	LoG > 0.045 at σ=1.4, pixel ratio	Textbook isotropic-blob filter at pore-sized scale.
Uniformity (inv.)	std(LAB.L) mapped 0–10, inverted	Low variance ⇒ uniform tone. The only metric where higher is better.

Each raw measurement is linearly clamped against a published constant (PIGMENTATION_RAW_RANGE, etc.). **Re-calibrating to a clinic's distribution = editing one tuple, not retraining a model.**

Reproducibility. Zero stochastic ops, zero LLM calls, zero network I/O in the scoring path. Same input → bit-identical output, enforced by tests/test_scoring_determinism.py (8 tests, including a no-input-mutation guard). Under mild perturbation (rotation ≤ 1.5°, exposure ±4 %, JPEG re-encode q = 82–95; 20 trials on one face), scripts/reproducibility_evidence.py yields $\bar{\sigma}(\text{ours}) \approx 0.074$ vs $\bar{\sigma}(\text{stochastic baseline at } \sigma=1.0) \approx 0.538$ — a 7× tighter band (per-metric: pigmentation 0.103, erythema 0.137, wrinkle 0.000, pore 0.000, uniformity 0.131). That gap is the difference between a longitudinal chart you can act on and one you can't. The right panel uses a *simulated* baseline (no live Vision-LLM credit was available in the 48 hr window); the script is parameterised so swapping in a Claude vision call regenerates it without other changes.



Explainability. The score formula *is* the explanation. "Why is pigmentation 7.2?" is answered with a heatmap of the black-hat response — the same intermediate the score is computed from — surfaced directly above the score card (visualization.py:metric_response_map). The history page re-uses the same composer behind flat toggles (Streamlit forbids nested expanders), so the doctor↔patient comparison stays interactive across visits.

Scores-only contract — the structural anti-"thin-wrapper" guard. The Explainer Protocol accepts a RegionScores dataclass + a short patient-context string — **never pixels, never the image path, never the QualityReport**. The factory get_explainer() resolves Anthropic / Gemini / Mock by the LLM_BACKEND env var or auto-selects (Anthropic → Gemini → Mock) based on which keys are present; both real backends try/except to MockExplainer on SDK error, so the demo never hard-fails. Because the contract is *structural* (the Protocol's explain() signature has no image parameter), no code path lets the LLM synthesise a numeric score — even if a future contributor wanted it

to. **Extensibility:** each scoring function is `bgr → float`; a new procedure-specific metric (volume change for filler, vascularity for rosacea) is one function plus one aggregator entry. No retraining, no schema migration. PRD §5's "next procedure" is a plug-in, not a rewrite.

4. Photo-Consistency Gate — the depth area · `consistency_gate.py`

Four checks gate every intake photo before scoring:

- 1. **Pose.** Decompose MediaPipe's 4x4 facial-transformation matrix into yaw / pitch / roll (ZYX Euler). Reject if any axis exceeds `POSE_TOLERANCE_DEG = ±15°` in frontal mode; profile mode uses `PROFILE_YAW_MIN_DEG = 5°` for side captures. The threshold was relaxed from a DSLR-tuned $\pm 8^\circ$ once live webcam capture went in — natural laptop-camera tilt puts roll at -8° to -10° , so $\pm 8^\circ$ was unreachable without forcing unnatural, rigid intake photos (see `BUILD_NOTES`).
- 2. **Exposure.** Fraction of grayscale pixels < 10 (under) and > 245 (over). Reject if either $> 2\%$ or mean brightness $< 60 / > 210$.
- 3. **Sharpness.** Laplacian variance on the **face-bbox crop** (full-frame fallback when no landmarks are detected, so the offline blur regression test still trips); reject if $< \text{SHARPNESS_MIN_LAPLACIAN_VAR} = 30$. Lowered from a DSLR-tuned 80 once Mac-webcam JPEGs were measured — well-lit frames land at 15–20 on the full frame, 35–80 on the face crop. The blurry-image regression test (`GaussianBlur(51, 25)`) still trips at the new floor.
- 4. **Color calibration.** Detect ArUco 5x5 markers (`DICT_5X5_50`); if present, sample the printed gray surround, compute per-channel gains, apply to the full image. If absent, **warn rather than hard-reject** — clinics adopt the calibration card progressively, so we degrade gracefully.

Failures yield actionable reasons (rendered to the receptionist in Traditional Chinese in the UI; the English equivalent here is "head turned 18.4° right; tolerance ±15°; please face the camera"); the full `QualityReport` is JSON-serialised onto the `Visit` row for audit. **Live-capture variant** (Session 2): a custom Streamlit component runs the same model **in the browser** via the Tasks Vision Web SDK and auto-captures only when all four checks pass for `LIVE_CAPTURE_STABILITY_FRAMES = 10` consecutive frames — converting the gate from "reject after the fact" to "guide in real time". Face-fill is bounded to `[0.35, 0.75]` of frame width so pore / wrinkle metrics have enough pixels without clipping the chin. **The server-side gate still re-runs on the captured frame** — the browser HUD is a UX accelerator, not a security boundary.

5. Workflow integration

Three-actor flow: receptionist captures (live or upload) → server-side gate runs → on pass, 5 metrics × 4 ROIs + LLM draft are produced → physician (sitting next to the receptionist) reviews the visit-history page with the ROI overlay toggle, edits the treatment-plan draft, saves → longitudinal page updates. The patient sees the heatmap + explanation on-screen during the consult.

Streamlit pages (6, sidebar nav defined in `app.py::PAGE_LABELS`):

Key	Label (UI)	Owns
patients	Patient management	CRUD + soft-delete + restore
intake	New visit	Live face-mesh capture + upload fallback + gate + scoring + LLM draft + save
history	Visit history	Per-visit timeline + ROI overlay toggles + CLAHE thumbnails
treatment	Treatment plan	Editable plan tied to the latest visit
overview	Longitudinal tracking	Radar chart (current) + line chart (all visits × 5 metrics)
settings	Settings	Gate thresholds, LLM backend status, audit toggles

Stack rationale. Streamlit (single-file UI, one-click Cloud deploy — the right tool for a 48 hr CV-pipeline demo; React + FastAPI is a 1-week refactor, deferred). Where Streamlit was too restrictive (no nested expanders, no client-side ML) we dropped to a `declare_component` widget for live capture — the Python side just consumes `{front, left?, right?}` and the rest of the pipeline is unchanged. `SQLModel` (same `BaseModel` API as the rest of the codebase; single `SQLite` file ships in the repo). Migrations are zero-downtime `ALTER TABLE ADD COLUMN`. State lives on disk — no Redis, no workers, no queues — the entire pipeline runs synchronously in the request thread because the slowest stage (MediaPipe) is < 200 ms on Apple Silicon. History-view hot paths use `@st.cache_data` keyed on `(path, mtime)` so toggling the ROI overlay does not re-run MediaPipe.

6. Stack, cost, latency · `scripts/benchmark.py` (M4 Pro, 3 faces × 5 runs)

Stage	p50	p95
Alignment + ROI extraction (MediaPipe Tasks)	7.0 ms	7.5 ms
Consistency Gate (4 checks + ArUco)	8.1 ms	8.5 ms
Scoring (5 metrics × 4 ROIs)	2.2 ms	2.5 ms
End-to-end (excl. LLM)	17.2 ms	18.1 ms
Explainer (Claude Sonnet 4.6, network)	~1.5 s	~2.5 s

Python 3.11 (mediapipe 0.10 has spotty 3.12 wheels on macOS arm64). uv-managed deps, ruff-formatted, pytest-tested (56 tests across 7 files). anthropic + google-genai SDKs are optional; absent both, the app runs against `MockExplainer` and the loop still works end-to-end. Per-visit LLM cost at Sonnet 4.6 pricing ≈ **\$0.0048 / visit** (~600 in + ~200 out tokens); 1 000 visits / month ≈ **\$5 / month** — a rounding error vs. the human time saved drafting the treatment note.

7. Limitations · full catalogue in `docs/LIMITATIONS.md`

Scoring ranges are calibrated on 3 reference photos (a real pilot would re-fit on ~200 per Fitzpatrick type). No identity confirmation on intake (Phase 2: face-embedding vs. the patient's first-visit photo). ArUco card adoption is voluntary (Phase 2: required-marker clinic setting). No HIPAA / PIPL story — photos sit in clear on disk (Phase 2: at-rest encryption + a deletion API). Skin-surface visibility is not validated: heavy makeup, partial occlusion, and smartphone beauty filters can pass every gate check and still produce misleading scores. **What was reused vs. built** is documented in `docs/BUILD_NOTES.md` per the brief.